Сценарий 7. Курсы валют: обновление по расписанию и допустимое устаревание.

Тест: `src/test/java/io/github/olaaa/distributedcachelab/scenario/Scenario7ExchangeRatesTest.java`. Сервис: `ExchangeRateService` (`REFRESH_AFTER` = 5 минут, `MAX_AGE` = 1 сутки, последние известные курсы — ключи `rates:last::<валюта>`). Прогрев: `ExchangeRateWarmUp`. Поставщик: `RateProvider` — имитация внешнего поставщика курсов. Валюты: `rates.currencies` = USD, EUR, CNY. Ручная проверка: `http/07-exchange-rates.http` на двух репликах.

Коротко:

- **Что за данные.** Курсы валют: поставщик публикует их по своему расписанию, читают их на каждой странице с ценами. Курс пятиминутной давности годится, суточной — уже нет.
- **Прогрев.** При старте реплика загружает все курсы до того, как начнёт принимать запросы: в `ApplicationRunner`, а не в `@PostConstruct`.
- **Обновление.** Caffeine с `refreshAfterWrite`: через 5 минут курс перезагружается в фоне, а запрос сразу получает прежний.
- **Поставщик недоступен.** Каждый полученный курс хранится в Redis как последний известный. Пустой кэш — например, у новой реплики — заполняется оттуда. Неудачное обновление оставляет прежний курс.
- **Когда к поставщику ходят.** Не по часам: при старте реплики и при первом запросе после 5 минут с прошлой загрузки. Нет запросов — нет и обращений к поставщику. Подробно — раздел «Когда забираются курсы».
- **Предел.** Курс старше суток не отдаётся: такой запрос ждёт обновления, а если обновить не удалось — 503.

Ситуация.

Каталог показывает цены в долларах, евро и юанях. Курсы приходят от внешнего поставщика: он медленный, у него может быть лимит запросов, иногда он недоступен. Курс меняется несколько раз в день, и показать курс, полученный пять минут назад, — нормально. Ходить к поставщику на каждый запрос — нельзя.

У таких данных три особенности, от которых зависит устройство кэша:

1. Их мало и они известны заранее — три валюты. Их можно загрузить целиком при старте.
2. Они допускают устаревание. Поэтому истекать им незачем: лучше обновлять в фоне и пока отдавать прежние.
3. Источник ненадёжен. Значит, нужен ответ на вопрос, что отдавать, когда его нет.

Когда забираются курсы.

«По расписанию» в названии сценария — это расписание поставщика: он публикует новые курсы, когда публикует, раз в день или несколько раз в день. Мы его расписание не знаем и не подстраиваемся под него. Сами мы забираем курсы не в какое-то время суток, а в двух случаях:

1. **При старте реплики** — прогрев, все валюты сразу.
2. **При первом запросе к курсу, которому больше 5 минут** (`REFRESH_AFTER`). Этот запрос сразу получает прежний курс, а новый загружается в фоне; следующие запросы получат уже новый.

Таймера нет. Нет запросов — к поставщику никто не ходит, и курс в кэше стареет. Отсчёт 5 минут свой у каждой валюты и у каждой реплики: от момента, когда курс этой валюты последний раз записан в кэш этой реплики.

Пример — одна реплика, одна валюта:

Время	**Что происходит**	**Обращение к поставщику**
09:00	реплика стартует, прогрев	да — курс на 09:00
09:03	запрос	нет: курс из кэша, ему 3 минуты
09:07	первый запрос после 09:05	да, в фоне; сам запрос получает курс на 09:00
09:07, через 50 мс	следующий запрос	нет: в кэше уже курс на 09:07
21:58	последний запрос вечером, курс обновлён	да
22:00 – 08:00	запросов нет	нет: обновлять некому
08:00	первый запрос утром	да, в фоне; сам запрос получает курс на 21:58 — ему 10 часов, это меньше суток
с субботы до понедельника	запросов нет больше суток	нет
понедельник, 09:00	первый запрос	да, и запрос ждёт его: курс старше суток не отдаётся

Отсюда два следствия:

- **Насколько отстаёт курс при постоянном трафике.** Поставщик опубликовал новый курс — он попадёт в кэш не позже чем через 5 минут после прошлого обновления, плюс время до первого запроса после этого срока. Если запросы идут постоянно, отставание — до 5 минут.
- **Насколько отстаёт курс, если запросов мало.** Сколько угодно — до первого запроса. Поэтому предел `MAX_AGE` и проверяет возраст курса при каждом ответе.

Если поставщик публикует курсы в известное время — например, в полночь, — их можно забирать по расписанию: задача `@Scheduled(cron = …)` сразу после публикации вызывает `refresh` для всех валют. Тогда утренний запрос получит уже новый курс, а не вчерашний с обновлением в фоне. На нескольких репликах — либо каждая сама, либо одна под распределённой блокировкой (redis-patterns-lab, сценарий 3) кладёт курсы в Redis, а остальные берут их оттуда. В лаборатории это не реализовано.

Прогрев при старте.

Без прогрева первые запросы к новой реплике ждут поставщика — или получают ошибку, если он недоступен. Поэтому курсы загружаются при старте, в `ExchangeRateWarmUp`:

```
@Component
public class ExchangeRateWarmUp implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments arguments) {
        Long loaded = exchangeRateService.warmUp().block();
        log.info("Прогрев курсов: загружено валют — {}", loaded);
    }
}
```

Почему `ApplicationRunner`. Документация Spring Boot: «Приложение считается готовым, как только вызваны раннеры приложения и командной строки». Пока раннеры работают, состояние готовности (readiness) говорит платформе не направлять запросы на эту реплику. И прямо: «Задачи, которые должны выполняться при запуске, нужно выполнять в компонентах `CommandLineRunner` и `ApplicationRunner`, а не в колбэках жизненного цикла вроде `@PostConstruct`». Kubernetes по пробе готовности не отправит трафик на реплику, пока курсы не загружены.

Что делать, если при старте курс не загрузился, — решение. Здесь прогрев пропускает недоступные валюты, и приложение стартует: эти курсы загрузятся при первом запросе или вернут 503. Другой вариант — уронить запуск, но тогда при недоступном поставщике новые реплики не поднимутся вовсе.

Обновление в фоне: `refreshAfterWrite`.

```
this.rates = Caffeine.newBuilder()
        .ticker(cacheTicker)
        .refreshAfterWrite(REFRESH_AFTER)
        .buildAsync(loader);
```

Документация Caffeine: «обновление ключа загружает новое значение асинхронно», и «пока ключ обновляется, по-прежнему возвращается старое значение (если оно есть)». Обновление запускается не по таймеру: «обновление фактически начинается только тогда, когда к записи обращаются». Через 5 минут после записи первое обращение получает прежний курс и запускает перезагрузку; следующие получат новый.

`expireAfterWrite` здесь нет нарочно: курсу не нужно исчезать из кэша, ему нужно обновляться. Предел устаревания проверяется отдельно, по `MAX_AGE`. Подробно про `refreshAfterWrite` и `expireAfterWrite` — сценарий 7 caching-lab, «Refresh-Ahead».

Когда поставщик недоступен.

Последний известный курс — в Redis. Каждый курс, полученный от поставщика, сохраняется в Redis без срока жизни: `rates:last::USD` → стрелка `{"currency":"USD","rate":92.50,"asOf":"…"}`. `asOf` — момент, на который поставщик выдал курс.

Первая загрузка — локального курса ещё нет: новая или перезапущенная реплика. Поставщик недоступен — курс берётся из Redis. Без Redis такой реплике отдавать было бы нечего. Нет и в Redis — ошибка, ответ 503.

Фоновое обновление — локальный курс есть. Caffeine: «если при обновлении брошено исключение, старое значение сохраняется, а исключение записывается в журнал». Но перед ошибкой загрузчик заглядывает в Redis: если там курс новее локального — его успела получить другая реплика, — берётся он.

```
private Mono<ExchangeRate> reload(String currency, ExchangeRate oldRate) {
    return fetchAndRemember(currency)
            .onErrorResume(RateProviderUnavailableException.class, unavailable ->
                    lastKnown(currency)
                            .filter(lastKnown -> lastKnown.asOf().isAfter(oldRate.a
sOf()))
                            .switchIfEmpty(Mono.error(unavailable)));
}
```

Почему не вернуть из Redis тот же самый старый курс как «успешное» обновление. Caffeine засчитал бы обновление и не пробовал бы снова ещё 5 минут: поставщик вернулся, а реплика всё это время отдаёт старый курс. С ошибкой Caffeine оставляет прежнее значение и пробует обновить при следующем обращении. В первой версии сценария загрузчик так и делал; ошибка найдена до первого прогона тестов.

Предел устаревания. Курс старше `MAX_AGE` = 1 сутки не отдаётся. Но старый курс в кэше — не всегда беда поставщика: может быть, к курсу просто больше суток никто не обращался, и обновлять его было некому (последняя строка примера выше). Поэтому такой запрос не получает 503 сразу, а ждёт обновления — как при промахе:

```
private Mono<ExchangeRate> awaitRefresh(String currency, ExchangeRate tooOld) {
    return Mono.fromFuture(() -> rates.synchronous().refresh(currency), true)
            .onErrorResume(refreshFailed -> Mono.empty())
            .defaultIfEmpty(tooOld)
            .flatMap(rate -> isFresh(rate)
                    ? Mono.just(rate)
                    : Mono.error(new StaleRateException(currency,
Duration.between(rate.asOf(), clock.instant()))));
}
```

`get` уже запустил фоновое обновление, и `refresh` возвращает его же. Javadoc `LoadingCache.refresh`: «если другой поток уже загружает значение для ключа, возвращается существующий future, и больше ничего не делается». Второго запроса к поставщику нет. Обновить не удалось или и новый курс слишком старый — `StaleRateException`, ответ 503.

В первой версии сценария проверка возраста сразу отвечала 503. Ошибка нашлась, когда пример выше расписали по времени: после выходных без запросов первый запрос в понедельник получил бы 503 при работающем поставщике.

Сколько устаревания допустимо — решение бизнеса. Для витрины можно отдавать и старый курс с пометкой даты, для расчёта оплаты — нет.

Ошибка не кэшируется. Упавшую загрузку Caffeine убирает из кэша сам, следующий запрос попробует снова.

Две гонки, которые поймали тесты

Обе — та же особенность, что в сценарии 3 и в caching-lab: Caffeine узнаёт о завершении загрузки колбэком `CompletableFuture`, а колбэки выполняются в обратном порядке регистрации. `block()` в тесте возвращает результат раньше, чем отработает колбэк Caffeine.

- **Время записи.** Тест загрузил курс и сразу сдвинул `ManualTicker` на 5 минут. Caffeine записал время уже после сдвига, запись «помолодела», и обновление не запускалось. В одиночном прогоне тест проходил, в прогоне всего класса — падал. Исправление в тесте: перед сдвигом часов дождаться, пока Caffeine запишет время (`awaitEntryWritten`).
- **Упавшая загрузка.** Поставщик вернулся, но следующий запрос сразу после ошибки получил тот же упавший future: Caffeine ещё не успел его убрать. Исправление в тесте: ждать нового курса через `await`, а не проверять сразу.

На работу приложения это не влияет: под нагрузкой часы не прыгают на 5 минут за миллисекунду, а запрос в те же микросекунды после ошибки получит ту же ошибку, что и до неё.

Цена и что можно иначе

- Каждая реплика обновляет каждую валюту сама: 10 реплик и 3 валюты — до 30 запросов к поставщику каждые 5 минут. Если у поставщика жёсткий лимит, обновление делают в одном месте: задача по расписанию под распределённой блокировкой (redis-patterns-lab, сценарий 3) кладёт курсы в Redis, а реплики берут их оттуда. Это вывод, в лаборатории не проверялся.
- Реплика, прогретая из Redis при недоступном поставщике, считает такой курс свежим для `refreshAfterWrite`: снова к поставщику она пойдёт только через 5 минут, хотя курс может быть старым. Это следует из устройства `refreshAfterWrite` — срок отсчитывается от записи в кэш, а не от `asOf`.

Что проверяют тесты

`warmUpLoadsAllRatesSoRequestsDoNotWaitForProvider`: прогрев — три обращения к поставщику, следующие запросы его не ждут; курсы сохранены в Redis.

`afterRefreshPeriodOldRateIsServedWhileNewOneLoadsInBackground`: поставщик опубликовал новый курс; через 5 минут первое обращение получает прежний 92.50, фоновое обновление — одно обращение к поставщику, дальше — 93.10, и он же в Redis.

`providerDownDuringRefreshKeepsLastKnownRate`: поставщик недоступен — после 5 минут отдаётся последний известный курс, без ошибки. Поставщик вернулся — курс сразу обновляется: неудачное обновление не засчитано.

`rateOlderThanMaxAgeIsRefused`: поставщик недоступен больше суток — `StaleRateException`. Поставщик вернулся — свежий курс.

`firstRequestAfterQuietPeriodLongerThanMaxAgeWaitsForFreshRate`: больше суток к курсу никто не обращался, поставщик доступен — первый запрос дождался обновления и получил свежий курс. Обращений к поставщику — одно на загрузку и одно на обновление: `refresh` вернул обновление, которое запустил `get`.

`newReplicaStartsFromLastKnownRatesWhenProviderIsDown`: реплика A прогрелась, поставщик упал; новая реплика B прогревается — все три курса из Redis.

`replicaPicksUpNewerRateFromRedisWhenProviderIsDown`: B успела получить новый курс и сохранить в Redis, потом поставщик упал; A при обновлении не достучалась до поставщика, но взяла из Redis курс новее своего.

`withoutLastKnownRateAndProviderDownRateIsUnavailable`: ни поставщика, ни Redis — прогрев не уронил запуск, курс недоступен; поставщик вернулся — курс есть.

Сроки Caffeine и возраст курса двигает один `ManualTicker`: в тестах часы приложения (`Clock`) идут от него же.

Тест сценария прогнан пять раз подряд, весь набор лаборатории, 30 тестов, — три раза, без падений.

Попробовать вручную

Первая реплика — как обычно, вторая — с уже недоступным поставщиком (`rates.provider.failing=true`):

```
mvn spring-boot:run
mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=8081 --
rates.provider.failing=true"
```

В журнале каждой реплики при старте — строка прогрева. Дальше `http/07-exchange-rates.http`: курс на обеих репликах, состояние поставщика, валюта, которой нет. Результат такого прогона:

```
8080: Прогрев курсов: загружено валют — 3.
8080 USD: {"currency":"USD","rate":92.50,"asOf":"2026-10-31T23:58:09.543577Z"}.
8080 provider: {"calls":3,"failing":false}.
redis: {"currency":"USD","rate":92.50,"asOf":"2026-10-31T23:58:09.543577Z"}.
8081: Прогрев курсов: загружено валют — 3.
8081 USD: {"currency":"USD","rate":92.50,"asOf":"2026-10-31T23:58:09.543577Z"}.
8081 provider: {"calls":3,"failing":true}.
8081 GBP: Поставщик курсов недоступен, курса GBP нет HTTP 503.
```

Реплика 8081 трижды не достучалась до поставщика, но прогрелась всеми тремя курсами из Redis: `asOf` у неё тот же, что получила 8080. Валюты, которой нет нигде, — 503.

Посмотреть последние известные курсы: `docker compose exec redis redis-cli GET rates:last::USD`. Поставщика на реплике можно «уронить» и «поднять»: `POST /lab/rates/provider/fail` и `POST /lab/rates/provider/recover`.

Что важно заметить.

- Данные, которые допускают устаревание, лучше обновлять в фоне (`refreshAfterWrite`), чем дать им истечь: запрос не ждёт источник.
- Прогрев — в `ApplicationRunner`: Spring Boot не объявит реплику готовой, пока он не закончился.
- Последнее известное значение во внешнем хранилище спасает реплику с пустым кэшем, когда источник недоступен.
- Неудачное обновление должно остаться неудачным: «успех» со старым значением отодвинет следующую попытку.
- Обновление по обращению не идёт по часам: без запросов курс стареет. Возраст проверяется при каждом ответе, а слишком старый курс сначала пробуют обновить.
- Устаревание допустимо, но не бесконечно: предел и понятная ошибка после него.

Как сказать на собеседовании.

«Курсы валют допускают устаревание, поэтому не даю им истекать, а обновляю в фоне: Caffeine с `refreshAfterWrite` — после срока первое обращение получает прежний курс, а новый грузится асинхронно. При

старте прогреваю все курсы в `ApplicationRunner` — Spring Boot объявит реплику готовой только после него. Каждый полученный курс кладу в Redis как последний известный: если поставщик лежит, новая реплика стартует с ним, а неудачное обновление просто оставляет прежний курс. Ставлю предел — например, сутки: курс старше не отдаю — жду обновления, а не вышло — отвечаю 503. Обновление по обращению не идёт по часам, поэтому если поставщик публикует курсы в известное время, добавляю обновление по расписанию сразу после публикации. Если у поставщика жёсткий лимит, обновляю курсы в одном месте по расписанию под блокировкой, а реплики читают из Redis».

Источники

- Spring Boot — SpringApplication: Application Availability: «Readiness State», события запуска.
- Caffeine — Refresh.
- Caffeine 3.2.4 — javadoc `LoadingCache.refresh`; исходники: `BoundedLocalCache.refreshIfNeeded` — неудачное обновление восстанавливает время записи, и следующее обращение снова запускает обновление.